Static InvertedBlock Indexing and Parameterized Search for Approximate MIPS over Learned Sparse Representations

Roger Canal Estrada

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
roger.canal@estudiantat.upc.edu

Abstract. Learned Sparse Representations (LSR), such as SPLADE, preserve lexical interpretability but create high Maximum Inner Product Search (MIPS) latency in very high-dimensional sparse spaces. This paper presents a static `InvertedBlock` indexing architecture with two phases: a high-capacity static index partitioned by document clustering, and a parameterized online search engine based on Coordinate-at-a-Time (CaaT) traversal. The static phase preserves candidate coverage, while the dynamic phase controls the latency–recall trade-off through runtime constraints. On Natural Questions (2.68M documents, 30,522 dimensions), the system reaches Recall@30 ≥ 0.90 at around 40 ms average latency and exposes direct search-time tuning levers without index rebuilding.

Keywords: Learned Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Inverted Indexing

1 Introduction

LSR models map semantics into vocabulary space, so retrieval remains sparse and interpretable [**?**]. However, LSR query weights do not follow the same heavy-head Zipfian pattern that classical lexical pipelines exploit. Instead, many coordinates carry moderate but non-negligible mass (often described as *wacky weights*) [**?**]. This geometry weakens the pruning selectivity of traditional WAND-like and Document-at-a-Time (DaaT) traversal because partial-score upper bounds become less decisive at early stages [**?**].

This work addresses this limitation with block-level reasoning. The key idea is to decompose each posting list into geometrically coherent blocks and score blocks through tight upper bounds before touching documents.

The paper contributes three elements. First, it defines a static `InvertedBlock` architecture where a high-capacity index preserves coverage and a parameterized search loop controls runtime cost. Second, it enforces exact-maximum summary bounds at index construction, making block upper bounds mathematically safe for aggressive skipping. Third, it shows that this separation reaches a stable operating point above Recall@30 = 0.90 while preserving independent latency control at query time.

2 System Architecture

The system is built around a custom partitioned inverted index designed to support efficient approximate Maximum Inner Product Search over sparse vectors. The central goal of this data structure is to organize document information so that the search engine can quickly identify the most similar documents to any incoming query without scoring every document individually.

The pipeline has two phases: static `InvertedBlock` construction (Section **??**) and parameterized query-time search (Section **??**).

2.1 Static Index Phase

The static pipeline has three steps. First, sparse K-means assigns each document to exactly one cluster. Second, for every cluster and vocabulary term, the index computes exact maximum term weights to build dense summary vectors. Third, for each term, it keeps only the top-nb clusters and then the top-nd documents per retained cluster, producing the final `InvertedBlock` lists used at query time.

Document Clustering Step. The method partitions the document set $\mathcal{D} = \{d_i\}_{i=1}^{N}$ into K clusters by a sparse K-means objective:

$$\max_{\{\mathcal{C}_k\},\{\mu_k\}} \sum_{k=1}^{K} \sum_{d_i \in \mathcal{C}_k} \langle d_i, \mu_k \rangle, \tag{1}$$

with assignment step

$$a(i) = \arg \max_{k \in \{1,\ldots,K\}} \langle d_i, \mu_k \rangle, \tag{2}$$

and centroid update

$$\mu_k \leftarrow \frac{\sum_{d_i \in \mathcal{C}_k} d_i}{\left\| \sum_{d_i \in \mathcal{C}_k} d_i \right\|_2}. \tag{3}$$

Both operations are computed in sparse form. This is a *hard clustering*: each document d_i is assigned to exactly one cluster $\mathcal{C}_{a(i)}$, and no document belongs to more than one cluster.

Partitioned Inverted Index Construction. The index maps each concept t to a list of `InvertedBlock`s. Each block stores one cluster id and truncated doc-weight pairs. For each cluster c, the method also builds a dense summary vector $S_c \in \mathbb{R}^{|V|}$ with

$$S_c[t] = \max_{d \in \mathcal{C}_c} d[t]. \tag{4}$$

This is the absolute maximum weight of concept t inside cluster c.

Algorithm 1 Static Index Construction with Exact-Maximum Summaries

Require: Sparse documents, cluster assignments, nb, nd
Ensure: Partitioned index I and summary vectors S

for each cluster c **do**
  initialize dense summary $S_c \leftarrow 0$
  for each document $d \in \mathcal{C}_c$ **do**
    for each non-zero (t, w) in d **do**
      $S_c[t] \leftarrow \max(S_c[t], w)$
    end for
  end for
end for
for each concept t **do**
  aggregate candidate docs by cluster and concept weight
  keep top-nb clusters by accumulated weight on t
  for each retained cluster c **do**
    keep top-nd docs by weight on t
    append `InvertedBlock`$(c, doc\_ids, weights)$ to $I[t]$
  end for
end for
return I, S

For each concept t, the index keeps at most nb cluster blocks and at most nd documents per retained block. Exact maxima make the cluster upper bound mathematically valid (never under-estimated).

$$UB_c(q) = \sum_{t \in \text{supp}(q)} q[t] \cdot S_c[t] \tag{5}$$

This property prevents miss-skipping. If summaries under-estimate, the system can discard clusters that contain true top-k documents.

2.2 Search Phase

Coordinate-at-a-Time (CaaT) Traversal The search engine processes incoming sparse queries over the static index with CaaT traversal. It parses non-zero query coordinates, sorts them by descending weight, and enters the inverted lists of each coordinate in turn.[1]

Algorithm **??** formalizes the CaaT loop.

Aggressive Dynamic Pruning Heuristics The pruning stage rejects blocks by the inequality

$$UB_c(q) < H.\min() \cdot heap\_factor. \tag{6}$$

[1] CaaT is preferred over classical Document-at-a-Time (DaaT) traversal [**?**] because it visits the highest-signal query coordinates first, rapidly inflating the min-heap threshold. A high threshold amplifies subsequent block-skipping decisions. DaaT, by contrast, mixes low- and high-signal evidence across all documents before the threshold stabilizes, reducing early pruning power.

Algorithm 2 Parameterized CaaT Search

Require: Query q, index I, summaries S, top-k, max_query_terms, max_search_blocks, $heap\_factor$, md
Ensure: Top-k result heap H

$Q \leftarrow$ non-zero query coordinates sorted by weight descending
if $max\_query\_terms > 0$ **then**
    truncate Q to first max_query_terms terms
end if
compute $UB_c(q)$ for all clusters c
initialize min-heap H, visited set, $docs\_examined \leftarrow 0$
for each term $t \in Q$ **do**
    $blocks\_seen \leftarrow 0$
    for each block $b \in I[t]$ **do**
        if $max\_search\_blocks > 0$ and $blocks\_seen \geq max\_search\_blocks$ **then**
            break
        end if
        $blocks\_seen \leftarrow blocks\_seen + 1$
        if $UB_{b.cluster\_id}(q) < H.\min() \cdot heap\_factor$ **then**
            continue
        end if
        for each doc d in b **do**
            if d not visited **then**
                score $\leftarrow \langle q, d \rangle$ and update H
                mark visited; $docs\_examined \leftarrow docs\_examined + 1$
                if $md > 0$ and $docs\_examined \geq md$ **then**
                    return H
                end if
            end if
        end for
    end for
end for
return H

If the inequality holds, the block cannot improve the current top-k under the selected confidence multiplier.

This rule replaces expensive document-level scoring with $O(1)$ float comparisons at block granularity. The search engine exposes four hyperparameters that restrict traversal without changing the static index:

1. `heap_factor`: relaxation multiplier controlling skip confidence.
2. `max_query_terms`: truncates low-signal query tail coordinates.
3. `max_search_blocks`: limits per-coordinate block breadth online.
4. `max_docs_to_visit` (md): hard computational budget bounding worst-case latency.

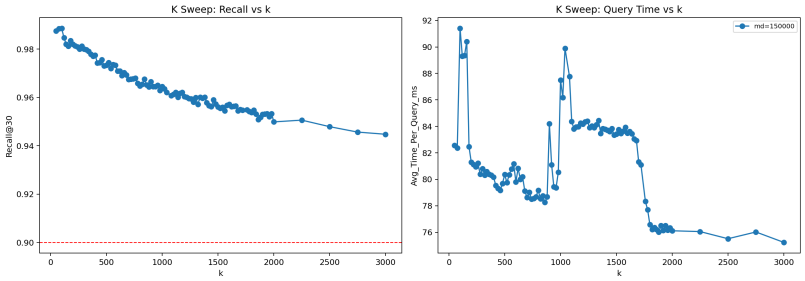

Fig. 1. Recall@30 and average query time as a function of K at three document budgets. $K = 1500$ consistently maximizes recall; larger K reduces latency at the cost of marginal recall loss.

3 Experimental Evaluation

3.1 Experimental Setup

The evaluation uses the SISAP 2026 Task 3 dataset: Natural Questions (NQ) with 2.68M documents encoded as SPLADE sparse vectors in 30,522 dimensions. Recall@30 is measured against the provided exact k-NN ground truth, and average query latency is reported. This protocol aligns with established LSR evaluation practice [**?**]. [2]

Experimental methodology. All experiments follow a staged wide-to-narrow sweep. First, the cluster count K is swept over a wide range. The best K found is then fixed while nb and nd are swept jointly. Finally, with the index frozen, the search parameters md, `heap_factor`, max_query_terms (mqt), and max_search_blocks (msb) are swept individually. Each stage uses the optimal values from the preceding stage as fixed constants, and a narrow refinement sweep is run around the best point.

Hardware. All runs execute inside a Docker container constrained to 8 virtual CPUs and 24 GB RAM with swap disabled. The host machine is an AMD Ryzen 5 5600X (6 cores / 12 logical processors, base 3.70 GHz, 32 MB L3 cache).

3.2 Static Index Capacity and the Recall Ceiling

The static index capacity determines the maximum recall achievable by any search strategy: any document excluded at build time (Alg. **??**) is permanently unrecoverable. The following two sections characterize how K, nb, and nd affect this ceiling.

Effect of Cluster Count K. A wide sweep evaluates K from 500 to 3000 (step 500), with fixed $nb = 500$ and $nd = 500$. Results are reported at three document budgets ($md = 50000$, 100000, and 150000). Figure **??** summarizes the trend.

$K = 1500$ yields the highest recall at every budget (0.816 at $md = 50000$, 0.841 at $md = 100000$, 0.843 at $md = 150000$). Smaller K reduces latency

[2] The results shown correspond to commit `96dbbe68adbd486f7cdf7a716f708b93f8a78479`.

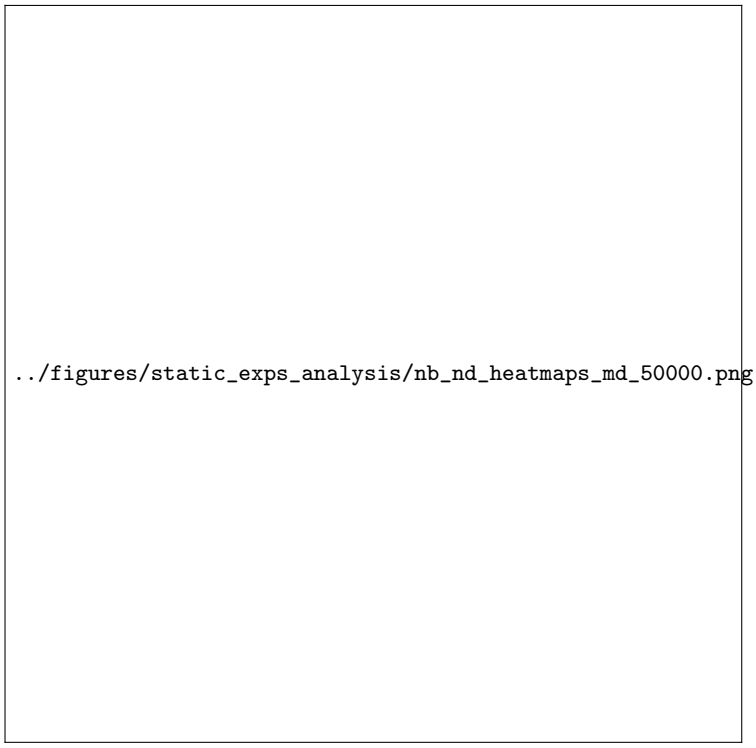

Fig. 2. Recall@30 heatmaps over nb and nd at $md = 50000$. Both axes raise the recall ceiling; the effect of nb is stronger at moderate md because more block entries survive the hard cap.

because cluster centroids are coarser and each concept's block list is shorter, but the recall ceiling drops. Larger K also reduces latency (fewer documents per cluster) while lowering recall because finer partitions produce smaller blocks and weaker candidates survive the nb/nd truncation. Based on this, $K = 1500$ is selected as the index constant for subsequent sweeps.

Effect of InvertedBlock parameters (nb) and (nd). With $K = 1500$ fixed, a joint sweep over $nb \in \{100, 500, 1000\}$ and $nd \in \{50, 150, 500\}$ is run at $md \in \{50000, 100000, 150000\}$. Figure **??** shows the recall heatmaps at $md = 50000$.

Both parameters raise the recall ceiling as more candidate evidence is retained in each term list. Increasing nd improves within-block coverage, while increasing nb expands the number of clusters that remain searchable per term. The gains are real but sublinear once the ceiling is already high. $nb = 500$, $nd = 500$ is selected as the operating default: it achieves Recall@30 = 0.922 at $md = 100000$ (54.8 ms) with the lowest build time among configurations above the task target, at 218 s. Doubling nb to 1000 adds only $\sim$0.002 recall while increasing build time by 24%.

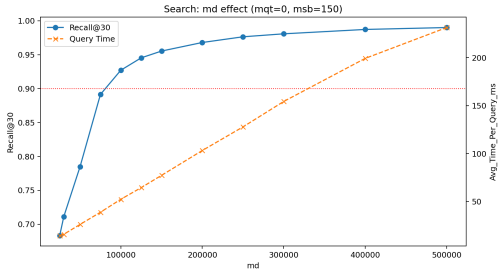

Fig. 3. Effect of *md* on Recall@30 and query latency. Both metrics grow monotonically. Recall@30 ≥ 0.90 is first met at $md = 100000$ (0.921 at 51.5 ms/q).

3.3 Search Parameterization and Latency Optimization

With the static index fixed at $K = 1500$, $nb = 500$, $nd = 500$, the search parameters are swept independently. Algorithm **??** governs all four parameters.

In this section, *md* denotes the *Maximum number of Documents* that can be fully scored per query.

Maximum number of Documents (*md*). *md* is the strongest recall lever. Figure **??** shows its effect from $md = 25000$ to $md = 150000$ across the tested budget points.

Recall and latency grow monotonically with *md* because the hard document cap is the binding stopping condition in every run. Average documents examined equals *md* in all cases, confirming that no block-skipping triggers before the budget is exhausted. *md* alone drives the recall–latency trade-off at these operating points.

Max Query Terms (mqt). mqt limits the number of highest-weighted query coordinates processed (0 = all). A sweep from mqt = 0 to mqt = 200 at $md = 50000$ and $md = 100000$ shows that recall and latency remain within ± 0.001 across the tested points (Figure **??**). For SPLADE queries, *md* is exhausted before low-weight coordinates add new candidates, so truncating the query tail has no measurable effect under this operating regime.

Max Search Blocks (msb). msb limits the number of `InvertedBlock`s evaluated per query coordinate. Unlike mqt, msb has a strong effect at low values (Figure **??**). At $md = 50000$, msb = 10 drops recall to 0.585; msb = 40 partially recovers to 0.766; msb ≥ 150 saturates and matches the unlimited baseline (0.770). The relevant candidate clusters for each concept span at least 150 blocks, so any msb below that threshold causes systematic miss-skipping and recall collapse.

Heap Factor. Across all tested configurations, `heap_factor` has no measurable effect on recall or latency. Because *md* is the binding stopping condition in

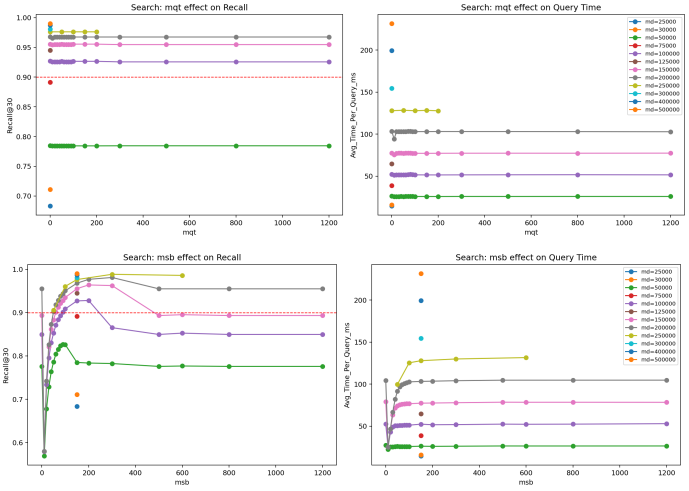

Fig. 4. Search controls at fixed index: top, mqt has near-zero effect because *md* binds first; bottom, msb strongly affects recall at low values and saturates for msb ≥ 150.

every run, the pruning inequality $UB_c(q) < H.\min() \cdot$ `heap_factor` is almost never triggered before the budget is exhausted. `heap_factor` therefore acts as a theoretical safety valve rather than a practical performance lever under the current operating regime.

4 Conclusion

This paper presents a mathematically grounded decoupled architecture that combines exact-maximum cluster summaries with CaaT traversal and block-level dynamic pruning. The static phase preserves a high-recall candidate pool, and the parameterized search phase provides direct runtime control of latency. Across the reported experiments, the `InvertedBlock` structure proves to be an effective data structure for approximate MIPS over LSR, since it consistently supports high recall while enabling practical latency control through online constraints.

Future work should further investigate this structure under broader workloads and larger tuning spaces to quantify its full potential and limits. Two immediate directions are automated search-parameter selection from query density statistics, and memory-aligned forward-index layouts to accelerate sequential access during exact sparse dot products.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., Zien, J.: Efficient query evaluation using a two-level retrieval process. In: *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)* (2003)

2. Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
3. Thakur, N., Reimers, N., Rückle, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: *NeurIPS Datasets and Benchmarks Track* (2021)
4. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021)
5. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024)
6. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Pairing clustered inverted indexes with κ-NN graphs for fast approximate retrieval over learned sparse representations. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)* (2024)